

TRABAJO GITHUB 003

JUAN SEBASTIAN RAMIREZ VELEZ

CENTRO DE TECNOLOGÍA Y MANUFACTURA AVANZADA

ANÁLISIS Y DESARROLLO DE SOFTWARE - FICHA 3169892

LUIS FERNANDO SÁNCHEZ PÉREZ

18/06/2025

ÍNDICE

Contenido

• ¿Qué es un repositorio en Git y cuál es su función dentro de un proyecto de desarrollo?	4
• ¿Qué problema resuelve el uso de sistemas de control de versiones como Git en proyectos de desarrollo de software?	5
• ¿Cómo permite Git la colaboración entre varios desarrolladores sin que sus cambios interfieran entre sí?	5
• ¿Cuál es la diferencia entre un commit y un push en el flujo de trabajo de Git?	5
• ¿Qué es un pull request en GitHub y cómo ayuda a revisar y fusionar los cambios de los colaboradores?	5
• ¿Qué ventajas ofrece el uso de ramas (branches) en Git cuando se está desarrollando una nueva funcionalidad o corrigiendo un error?	6
• ¿En qué situaciones se recomienda usar un fork en GitHub?	6
¿Cómo podría estructurar el flujo de trabajo del equipo utilizando Git y GitHub para asegurar que todos los miembros colaboren de manera eficiente y no se presenten conflictos de código?	7
Imagine que un miembro del equipo ha realizado cambios importantes en una funcionalidad de la evaluación de la rueda de la vida, pero al hacer un merge de la	8

rama en la que estaba trabajando, se presentan conflictos con la rama principal del	8
proyecto. ¿Cómo resolvería este conflicto utilizando Git y GitHub?	8
Como parte de su rol en el proyecto, decide automatizar las pruebas unitarias de las	8
funcionalidades de la web utilizando GitHub Actions. ¿Qué pasos seguiría para.....	8
configurar un flujo de trabajo en GitHub Actions que ejecute las pruebas	8
automáticamente con cada push a la rama principal y despliegue el sitio web de	8
manera continua?	8
ACTIVIDAD: CONFIGURACIÓN	9
EVIDENCIAS:	10
1.- Perfil:	10
2.- Mi organización:.....	11
3.- Mi proyecto.....	12
4.- Repositorio:	12

Preguntas de Introducción:

- **¿Qué es un repositorio en Git y cuál es su función dentro de un proyecto de desarrollo?**

Un repositorio es un directorio que contiene todos los archivos y carpetas de un proyecto, junto con el historial de cambios de cada archivo. Es donde Git almacena el código fuente y todas las versiones del proyecto a lo largo del tiempo. Un repositorio permite a los desarrolladores rastrear cambios, colaborar en el código y revertir a versiones anteriores si es necesario.

- **¿Qué problema resuelve el uso de sistemas de control de versiones como Git en proyectos de desarrollo de software?**

Los sistemas de control de versiones como Git resuelven problemas cruciales en el desarrollo de software, como el seguimiento de cambios, la colaboración eficiente y la gestión de versiones de código. Permiten a los equipos de desarrollo trabajar de forma más organizada y segura, evitando la pérdida de información, facilitando la colaboración y permitiendo la reversión a versiones anteriores del código en caso de errores.

- **¿Cómo permite Git la colaboración entre varios desarrolladores sin que sus cambios interfieran entre sí?**

Git permite la colaboración entre varios desarrolladores a través de un sistema de ramas y fusiones. Cada desarrollador trabaja en su propia rama, realizando cambios de forma aislada, y luego fusiona sus cambios con la rama principal cuando están listos, evitando interferencias.

- **¿Cuál es la diferencia entre un commit y un push en el flujo de trabajo de Git?**

En Git, un commit guarda cambios localmente en tu repositorio, mientras que un push transfiere esos cambios a un repositorio remoto. En resumen, el commit es para guardar el progreso local y el push para compartir ese progreso con otros.

- **¿Qué es un pull request en GitHub y cómo ayuda a revisar y fusionar los cambios de los colaboradores?**

Un pull request (PR) en GitHub es una solicitud formal para integrar cambios de una rama (generalmente, una rama de un colaborador) en otra rama (generalmente, la rama principal o

"main"). Actúa como una herramienta de colaboración, permitiendo a los desarrolladores revisar, discutir y aprobar los cambios antes de que se fusionen con el código base principal, mejorando la calidad del código y la gestión del proyecto.

- **¿Qué ventajas ofrece el uso de ramas (branches) en Git cuando se está desarrollando**

una nueva funcionalidad o corrigiendo un error?

- El uso de ramas (branches) en Git ofrece numerosas ventajas al desarrollar nuevas funcionalidades o corregir errores, principalmente porque permite trabajar en paralelo sin afectar la rama principal del proyecto y facilita la colaboración entre diferentes desarrolladores. Al crear una rama, se crea un entorno aislado donde se pueden realizar cambios, probar nuevas ideas o solucionar problemas sin riesgo de afectar la estabilidad del código base principal.

- **¿En qué situaciones se recomienda usar un fork en GitHub?**

Un fork en GitHub se recomienda principalmente para dos situaciones: contribuir a proyectos de código abierto donde no tienes permiso de escritura y crear tu propia versión personalizada de un proyecto existente.

Usted está trabajando en el desarrollo de una plataforma web para la empresa “Coquito

Amarillo S.A.S.”, cuyo objetivo es evaluar el nivel de satisfacción personal de sus empleados

a través de la “Rueda de la vida” (un enfoque utilizado en psicología para medir el bienestar

en diferentes áreas). La plataforma permite a los empleados completar un cuestionario

interactivo, visualizar los resultados y generar un informe con recomendaciones

personalizadas.

Durante el proceso de desarrollo, el equipo se enfrenta a varios desafíos relacionados con el control de versiones y la colaboración efectiva. El proyecto está siendo desarrollado por un equipo de varios miembros, y la coordinación de los cambios, la integración de nuevas funcionalidades y el despliegue del sitio web son aspectos críticos.

¿Cómo podría estructurar el flujo de trabajo del equipo utilizando Git y GitHub para asegurar que todos los miembros colaboren de manera eficiente y no se presenten conflictos de código?

R: Para asegurar una colaboración eficiente con Git y GitHub, cada miembro del equipo debe trabajar en su propia rama a partir de una rama principal de desarrollo (develop), subir sus cambios mediante pull requests, y asegurarse de actualizar su rama con frecuencia para evitar conflictos. El código debe ser revisado antes de integrarse a la rama principal (main). Nadie debería hacer cambios directos a main o develop; todo debe pasar por revisiones. Así se mantiene la estabilidad del proyecto y se facilita el trabajo en equipo.

Imagine que un miembro del equipo ha realizado cambios importantes en una funcionalidad de la evaluación de la rueda de la vida, pero al hacer un merge de la rama en la que estaba trabajando, se presentan conflictos con la rama principal del proyecto. ¿Cómo resolvería este conflicto utilizando Git y GitHub?

R: Para resolver el conflicto, el miembro del equipo debe primero asegurarse de estar en su rama de trabajo local, luego hacer un pull de la rama principal (main o develop) con `git pull origin main`, lo cual traerá los cambios y mostrará los conflictos. Debe abrir los archivos en conflicto, revisarlos manualmente y decidir qué partes conservar (lo editado por él, lo de la rama principal, o ambos combinados). Después de resolver, guarda los archivos, hace `git add .`, seguido de `git commit -m "resuelve conflictos"`, y finalmente `git push` para actualizar su rama en GitHub. Una vez hecho esto, el merge podrá completarse sin problemas.

Como parte de su rol en el proyecto, decide automatizar las pruebas unitarias de las funcionalidades de la web utilizando GitHub Actions. ¿Qué pasos seguiría para configurar un flujo de trabajo en GitHub Actions que ejecute las pruebas automáticamente con cada push a la rama principal y despliegue el sitio web de manera continua?

R: Para configurar GitHub Actions para pruebas y despliegue continuo, crearía un archivo YAML en `.github/workflows/` dentro del repositorio. En ese archivo, definiría un flujo de trabajo que se active automáticamente con cada push a la rama main. Primero, configuraría el entorno

(por ejemplo, Node.js si es una app web), instalaría las dependencias, ejecutaría las pruebas unitarias con un comando como `npm test`, y si todas pasan, ejecutaría el despliegue (por ejemplo, subiendo a un servidor, servicio como Vercel o GitHub Pages). Así garantizo calidad en el código y actualización continua del sitio sin intervención manual.

ACTIVIDAD: CONFIGURACIÓN

Para aprender debemos hacer, empecemos por realizar la configuración del repositorio y la estructura del proyecto en GitHub; para ello vamos a crear nuestro perfil de desarrollador y un repositorio en GitHub y configuraremos su estructura básica:

1. Crear su perfil de usuario en la plataforma GitHub (<https://www.github.com>).
2. Crear en su perfil una organización (Ej: Coquito Amarillo S.A.S)
3. Crear en su organización un proyecto (Ej: Web Corporativa)
4. Crear en su proyecto un repositorio con un nombre apropiado, como “coquito-amarillo-web”.
5. Clonar el repositorio en su máquina local utilizando ``git clone <URL del repositorio>``.
6. Crear la estructura inicial de directorios dentro del repositorio (por ejemplo, una carpeta ``documentación`` para la gestión administrativa del proyecto, ``src`` para los

archivos del código y una carpeta `assets` para imágenes, estilos y otros recursos).

7. Realizar el primer commit para guardar la estructura general y los archivos base del proyecto (Readme, Licenciamiento, Requerimientos, product backlog, HTML, CSS, imágenes).

8. Subir los cambios al repositorio remoto con el comando `git push`.

EVIDENCIAS:

1.- Perfil:

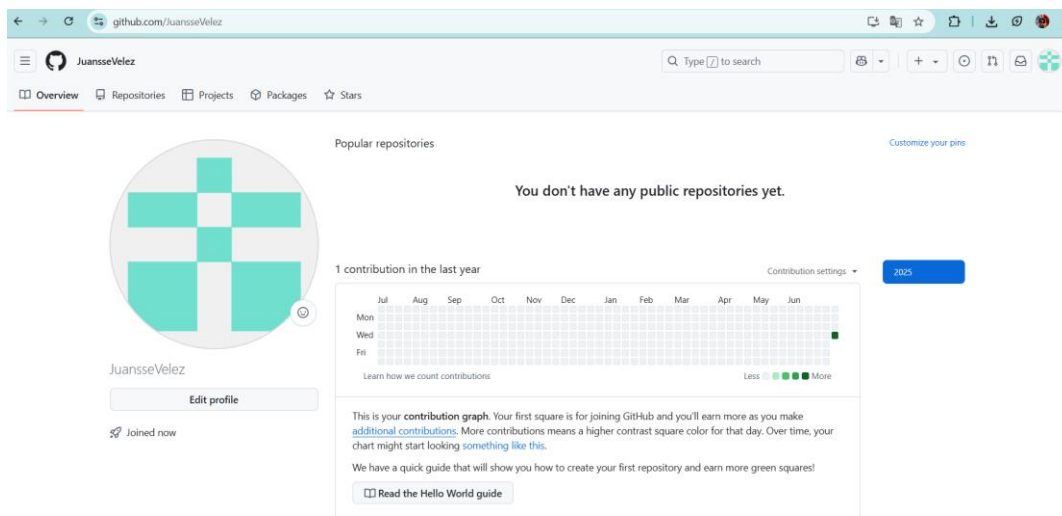


Ilustración 1 Github - Mi perfil

2.- Mi organización:

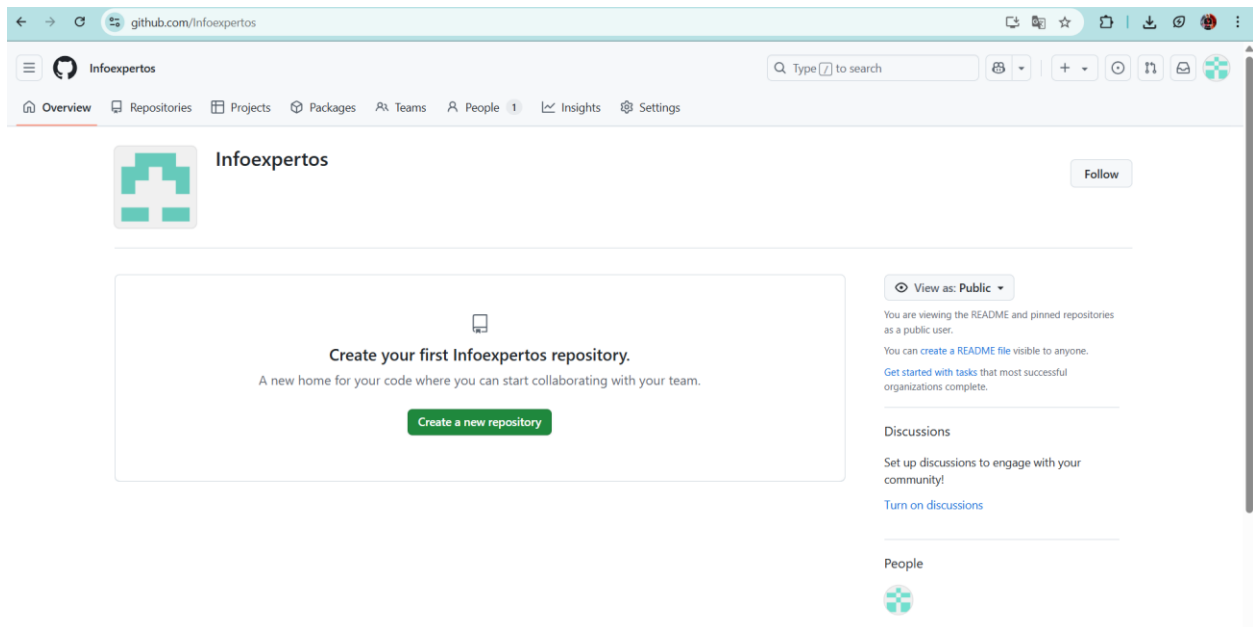


Ilustración 2 Github - Mi organización

3.- Mi proyecto

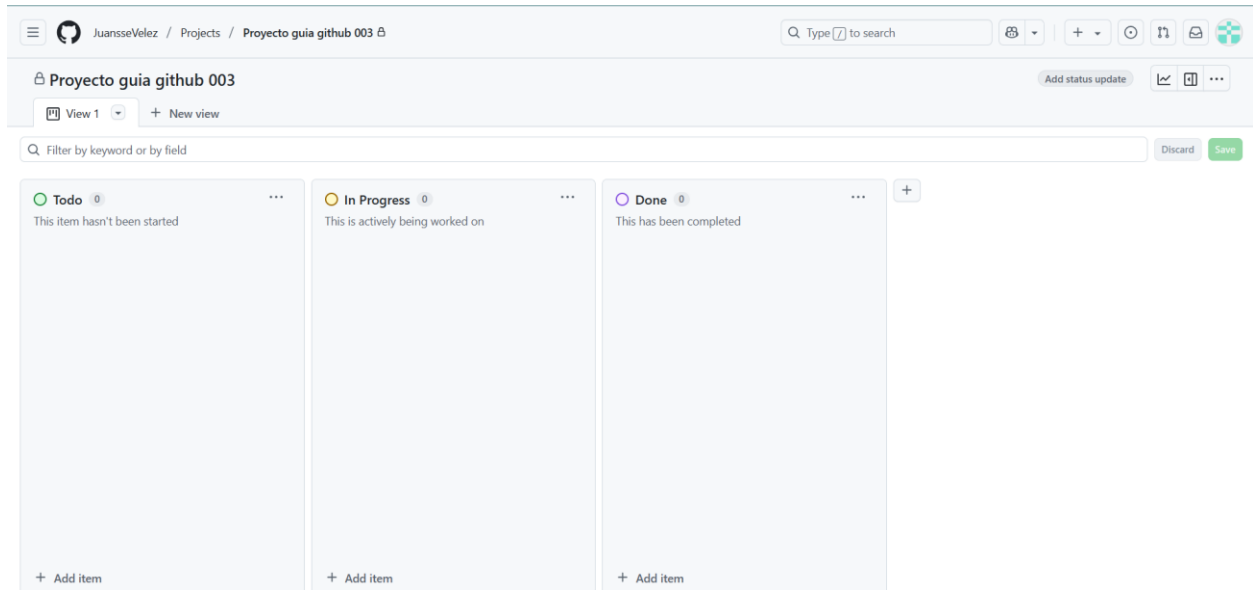


Figure 1 Github - Mi Proyecto

4.- Repositorio:

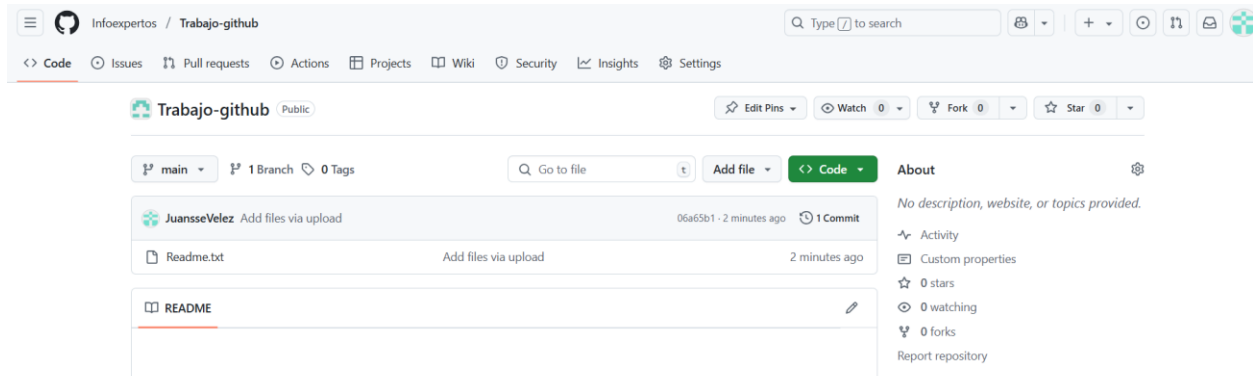


Figure 2 Github - Mi repositorio

